The Odyssey

A running log of the Safety-Dial project, one entry per task.

Sachin Mohan, supervised by Geraud Nangue Tasse.

Task 1: Replicate the ManiGuard Jar benchmark

Why. Before trying to improve the Jar policy we needed to know its real base rate, and whether our setup even matches the paper. The old runs never could: they used a different render profile (640-pixel high fidelity instead of the paper's 256-pixel legacy) on two tasks.

What we did.

- Pinned ManiGuard to the last revision that still renders the paper's profile, and ran the paper's protocol exactly: 26 Jar tasks, 2,000 steps, 8 of 16 actions, official evaluator.
- Rented one two-GPU machine on Vast and ran seeds 1 and 2, then seed 0 twice (one per GPU) to see how much a repeated seed moves.
- Dropped an earlier seed 0 run from a different machine so that everything of record comes from one host. With it the test still passed; without it the safe-success rate goes from 15% to 23%.
- Saved every rollout video to a private Hugging Face dataset so anyone can watch what the robot did. Total cost about $6.

Result. Counts over 26 tasks per run. The paper's row is for comparison.

	Safe success	Unsafe success	Safe fail	Unsafe fail	Violations
Seed 0, repeat 1	7	0	9	10	10
Seed 0, repeat 2	7	5	7	7	12
Seed 1	6	0	10	10	10
Seed 2	4	2	11	9	11
Ours, 104 episodes	24 (23%)	7	37	36	43 (41%)
Paper, 78 episodes	21 (27%)	4	27	26	30 (38%)

Fisher tests against the paper: $p = 0.60$ for safe success, $p = 0.76$ for violations. Our setup matches the paper.

What we learned.

- Running the same seed twice gives the same outcome on only 13 of 26 tasks, about the same as two different seeds. The robot reaches the jar the same way every time and then the physics at contact decides. One episode is a coin flip with a per-task bias.
- 13 tasks never succeed safely, 12 sit in the middle, one always succeeds. The middle 12 are where a dial could show anything.
- Most violations are the jar tipping over (33 of 43); the gripper only closed on the jar in 32 of 104 episodes.
- To tell a ten-point improvement from noise we will need about 200 episodes per condition. Everything after this is sized on that.

Where it lives. Videos and rows: `Sachioster/maniguard-jar-replication-20261022` (private). Receipts: `progress/receipts/`. Full write-up: *Jar replication and noise floor* under `docs/research/research_notes/`.

Task 2: Look at what actually happened

Why. Before trying any fix we wanted the 104 episodes to tell us where the leverage is, without assuming the answer. One rule for this task: describe, don't intervene.

What we did. An agent on a RunPod pod turned every episode into a timeline from the monitor logs (first contact, lid closed, lift, tip, violation), tiled frames from the videos at those moments for all 26 tasks, compared the two same-seed repeats task by task, and put successes next to failures of the same task to see where they first differ. It also wrote a per-step recorder for actions, joint state and jar pose, which the next GPU rental will switch on. The one simulator episode planned never ran: the pod's driver (595) crashes Isaac Sim's renderer at startup, so simulation stays on the 580-driver Vast hosts. Cost about $1.50.

What we found.

- The approach is the same every time. Repeats of one seed reach the jar within half a second of each other and look identical until the fingers meet the lid. Everything that decides the outcome happens after contact.
- Two moments decide it: whether the push gets the lid over, and the regrasp afterwards, when the gripper leaves the lid, descends, and lands on the body or on the rim. Seven of ten same-seed pairs split in that regrasp.
- Idling is as common as crashing: 36 episodes end with nothing lifted and no violation, half of them with the lid already closed and the robot parked. The biggest crash story is tipping the jar after closing it, 19 episodes.
- Only 8 tasks behave the same way every run (five always tip, two always park, one always succeeds). The other 18 are knife-edge.

Where that points. The lever is in execution and recovery after contact, not in the approach: a slower or shallower regrasp, a stop when a finger meets the rim, a retry when the push has stalled. The leverage note ranks three stage 2 experiments, each about 200 episodes per condition. Panels are frozen: development 0, 3, 6, 9, 15, 17; held out 2, 5, 8, 14, 16, 23, 7, 18, 12.

Where it lives. Repository, under `progress/receipts/`: the frame sheets (`02-frames/`), the pair strips (`02-pairs/`), the timelines, sensitivity map and task notes (`02-*.md`); the same sheets are in the dataset under `analysis/02/`. The leverage note, *Where the leverage is*, sits with the other research notes under `docs/research/`.

Task 3: The first knob, retry when the robot stalls

Why. Task 2 said the lever is after contact, and that a third of the episodes end with the robot parked. The simplest fix anyone would try is a retry: if the arm has stopped moving, let go, go back to where it started, and let the policy try again. Retry also has the best track record of the cheap fixes in the literature. Before evolving anything we wanted to know whether this one knob moves safe completion at all, and whether a scripted retreat tips the jar.

What we did.

- Built a small proxy between the simulator and the policy server. Off, it passes every byte through unchanged (checked). On, it watches the arm's joints and, if none has moved more than 0.01 rad in 4 seconds, replaces the policy's next four chunks with a scripted retreat: open the gripper, glide back to the first pose, hand control back. At most three times per episode. Policy and benchmark untouched.
- Froze the plan before the first episode: the six development tasks, seeds 100 to 115, every seed once with the proxy off and once on. 192 episodes, 96 pairs, two GPUs on one Vast host, 7 hours, about $6.
- Scored safe success and violations per arm, and a paired McNemar test over the 96 pairs.

Result. Counts over 96 episodes per arm.

	Safe success	Unsafe success	Safe fail	Unsafe fail	Violations
Native	17 (18%)	7	31	41	48 (50%)
Retry	16 (17%)	9	33	38	47 (49%)

Paired: retry minus native is -1 pair on safe success and -1 pair on violations, $p = 1.0$ for both. The knob, as frozen, changes nothing.

What we learned.

- The retreat fired in 21 of 96 retry episodes, and none of those 21 ended in success. Their native twins succeeded six times. The mechanism itself worked: the arm went home cleanly, no jar tipped during a retreat, and the policy resumed every time. It then parked again, or tipped the jar on the second attempt (5 of the 21). Resetting the arm does not reset the situation.
- It fired rarely because the stall test is wrong for this policy. A parked arm still jitters by 0.02 to 0.1 rad over 4 seconds, above our 0.01 threshold, so the detector only tripped when the gripper was pressed hard and motionless on the jar, the one stall a go-home cannot fix.
- The other 75 pairs are two runs of the identical episode, because the proxy did nothing in them. They agreed on safe success 85% of the time and on violations 64% of the time. The simulator's renderer is not reproducible across launches, so a same-seed pair shares its layout but never its pixels. That is the noise floor every later knob has to beat, and it rules out common random numbers for the search.
- Half of all episodes end in a violation, and the retreat never touched that. Stalls and tipped jars are different failures; this knob only ever addressed the first.

Where that points. A stall needs a definition this policy can satisfy, the jar not moving rather than the arm not moving, and the bigger prize is the violation rate. The next knob aims there.

Where it lives. Both arms and the receipts are in the same private dataset (`jar/gate03_native`, `jar/gate03_retry`, `receipts/03`). In the repository: plan, run log, scoring, identity check and frame sheets under `progress/receipts/03-*`; the proxy under `scripts/todo3/`.

Task 4: Do the native commands produce the requested motion?

Why. Task 2 showed where failures happen, but could not tell us whether the arm fails to follow its commands or follows unhelpful commands. That distinction matters before investing in an execution adapter inspired by the literature.

What we did. Froze twelve native episodes on development tasks 0, 3, 6, 9, 15 and 17 at seeds 300 and 301. Two GPU lanes split the work on one new Vast host, with the paper's control and rendering protocol. A passive recorder saved every proposed chunk, executed target, measured joint state, contact event, monitor trace and both camera videos. All twelve completed on their first attempt and passed alignment checks. We inspected every episode's joint plots and event frames, including successes and safe failures. No intervention or held-out task was used.

Result. Two safe successes, two unsafe successes, four safe failures and four unsafe failures. Safe requested completion was 2/12, violations 6/12 and engagement 12/12. Eight episodes never reached the goal. These are diagnostic counts on a different host, not a new pooled benchmark rate.

What we learned.

- Five of six violating episodes tracked closely in the two seconds ending at their first violation. Both unsafe successes carried a jar that never closed. Three safe unfinished episodes closed the lid but never grasped or lifted it. Missing continuation and poor interaction choices remain the broader explanation.
- One important exception, task 6 seed 301, developed a substantial discrepancy after closure and before tipping: mean post-action joint RMS error 0.0151 rad, against 0.000102 before contact. Physical joint limits do not explain that window. The trace cannot distinguish a causal tracking fault from resistance during an already bad contact.
- Large later errors often came from commands beyond joint limits. A separately labeled post-hoc source audit explains part, not all, of them. Increasing gain is not automatically the remedy. A successful episode also waited a long time after closure, so a slow interval alone is not a reliable failure detector.

Where that points. No general tracking deficit is established, but the contact exception keeps a bounded adapter hypothesis open. Any adapter comparison remains exploratory and must demonstrate safe completion, not just smaller tracking error. Command and continuation steering remains the stronger broader direction. We still need a working lever before an evolutionary claim.

Cost and record. The two-lane queue took 42 minutes after setup: 20,121 policy controls, 120 warm-up controls and 2,520 model queries including four model-only probes. Every recording was copied locally and hash-checked. Receipts are `progress/receipts/04-tracking-*`; full evidence belongs under `recordings/tracking04`, `jar/tracking04_native` and `receipts/tracking04` in the private dataset. The preservation receipt records the pinned revision and final rental state.

Task 5: Can bounded execution feedback improve safe completion?

Why. Task 4 found no general tracking deficit, but one contact-related discrepancy left a narrow adapter hypothesis open. We tested a concrete controller before allocating any evolutionary search budget.

What we did. Implemented three modes around the frozen Jar policy: exact native pass-through, fixed position-error feedback with gain 0.2, and an online adaptive correction using position error, measured velocity and reference acceleration. Corrections were bounded at 0.10 rad per arm joint; the gripper stayed native. The adaptive rule was our specified APEX-inspired discrete implementation, not verified author code or a reproduction of its stability guarantees. The controller saw no jar state or monitor propositions.

Two engineering episodes checked integration. The scored panel then ran all six development tasks at seeds 400 and 401 in every arm, twelve episodes per arm, balanced across two RTX 3090 lanes on machine 54514. Every one of the 38 starts completed and validated on its first attempt. No tuning, replacements, outcome-driven repeats or held-out tasks were used. Every recording's plots and event frames were reviewed.

Result. Counts over twelve scored episodes per arm. Violations here are the benchmark's counted violations.

	Safe success	Unsafe success	Safe fail	Unsafe fail	Violations
Native	1	1	6	4	5
Fixed	1	0	8	3	3
Adaptive	0	0	5	7	7

Safe requested completion was 1/12, 1/12 and 0/12. Unfinished episodes were 10/12, 11/12 and 12/12; engagement was 12/12, 12/12 and 9/12. Adaptive had eight raw violations: one tipped jar was excluded by the benchmark's contact gate. That nominal safe failure is not evidence of physical safety.

What we learned.

- Fixed and native succeeded safely on the same task but opposite seeds. Fixed's lower observed violation count did not bring a safe-completion gain. Twelve episodes per arm and six task clusters do not establish a population advantage or equivalence.
- Adaptive saturated 39.4% of joint corrections and produced about 15.5 times native's mean per-step joint motion, often starting before contact. It never registered a grasp. A bounded correction can still produce unhelpful oscillation and poor task behavior.
- Four native and four fixed safe unfinished runs closed the jar without ever grasping or lifting it. Task continuation remains a substantial failure mode. A quiet interval alone is insufficient to diagnose failure: the fixed success waited about 22 seconds between closure and grasp.

Where that points. This preset does not establish the useful lever needed before an optimizer comparison. Do not start evolutionary search on it. Any revised controller needs a separately frozen design and evidence of improved safe completion. This result concerns our implementation and settings, not all execution adapters or the original APEX method.

Cost and record. The scored queue took 89 minutes, with 17.5 minutes for engineering separately. Total accounting is 73,409 policy controls, 380 warm-up controls, 9,227 model-query attempts and 9,223 confirmed returns. Receipts are under `progress/receipts/05-adapter-*`; full evidence uses `recordings/adapter05`, `jar/adapter05_*` and `receipts/adapter05` in the established private dataset. The preservation receipt records verified revisions, provider charges and teardown.

Planning task: DSRL proposal and progress cleanup

Why. The retry and bounded-feedback presets did not establish a useful lever. Sachin selected DSRL, which learns input-noise steering around a frozen policy, as the next development experiment. He also selected a four-RTX-4090 host and requested a final plan review before implementation or rental.

What we did. Added a progress index and a receipt index, separated the superseded twelve-mechanism screen and its launch prompt from current work, and brought the first-principles status and agent map up to date. Completed protocols, rental records, panels and existing receipts remain at their recorded paths. The two superseded originals are retained unchanged, with notices at their old paths.

Read the authors' DSRL code at revision `7f48937` and finalized an integration stage followed by a smaller development pilot. Sachin selected a hard $15 total cap. The allocation is six engineering episodes, 24 training episodes and 48 evaluations across native, fixed-zero, bounded-random and learned-noise arms, with two infrastructure replacement starts reserved. Three rollout workers share one learner GPU during training; all four GPUs serve evaluation. At most $13 goes to active compute and disk, with $2 reserved for transfers and other storage. Aggregate rental time cannot exceed six hours or the money deadline, whichever comes first. Engineering must demonstrate that the fixed pilot fits the remaining allowance.

Review. The selected reward is one for strict safe requested completion and zero for other terminal outcomes, plus potential-based progress shaping. A fixed potential gives intermediate feedback for closure, closed-jar grasp, legal carry and the native goal predicate. It is zero at all episode endings; a numerical return identity checks that shaping does not add a net completion substitute. The potential scale is 0.25 and the per-control discount is 0.9998. Safety remains a separately logged cost and an outcome criterion. The final episode-24 checkpoint is selected in advance; the held-out tasks receive no new episodes. One learner seed and twelve evaluations per arm make this an exploratory pilot. A sparse-reward learner is deferred beyond this allocation. A positive result would justify replicated development work and stronger controls.

Independent review, 25 October. A workstation source and metadata audit confirms a 16 by 32 latent interface. The plan now freezes a 128-query magnitude-1 decoder probe, requiring robust gripper-command changes on two snapshots from two pool tasks before learning. Closure is latched once observed. The first attempt per GPU has a 40-minute wall timeout; later attempts retain 25 minutes. Engineering updates have their own ledger, and the authority documents agree that the full optimizer comparisons precede any evolutionary claim. Timeout termination is explicitly labeled as a departure from the author loop. Corrected discounted shaped return cannot rise from closure alone because the potential terms cancel. The $15 cap is unchanged.

Status and record. Cleanup and the reviewed plan are complete. The experiment has not been implemented, rented or run. Start at `progress/README.md`.

Plan: `progress/06-dsrl-plan.md`.
Review: `progress/receipts/06-dsrl-plan-review.md`.
Reviewer prompt: `progress/06-review-prompt.md`.

Task 6: DSRL implementation and engineering gates

Why. Test whether the frozen Jar policy offers a useful input-noise lever before spending the development allocation on ordinary learning. The reviewed plan required native replay identity, gripper authority and learner integration before training.

What we did. Implemented the explicit-noise server, passive transition and reward recorder, author-SAC bridge, deterministic queue, budget guards and private preservation. All 166 local tests passed. Rented four RTX 4090 GPUs on one new host and completed four native pool episodes. A grasp-enum validation bug was repaired against the already saved native predicate, preserving every raw value and outcome. No episode was replaced. Source freezes and the engineering measurement amendment record the changes.

Result. The 128-query decoder gate was invalid. Four of sixteen factual replay comparisons passed, all from lane 0; the other lanes differed by up to 0.00313 when replayed on lane 0. All sixteen retained the same converted gripper commands, and each pair of replays was exactly repeatable. Descriptive candidate sensitivity appeared on one of eight inputs, below the two-input, two-task minimum, but the invalid identity gate prevents an authority or absence conclusion.

GPU source-update checks also exceeded the unchanged 10^{-6} threshold by small optimizer-state differences. The 128 timed updates and remote CPU/GPU actor check did not complete. Engineering stopped at its frozen gate; no development training or evaluation ran. Safe-completion efficacy remains untested.

Accounting. Four starts, 8,000 policy controls, 40 warm-up controls, 1,140 confirmed VLA queries and eight engineering SAC calls, including two workstation calls. The four native pool episodes yielded strict safe completion 0/4, violations 2/4, unfinished 4/4 and engagement 4/4. There were no grasps. These are engineering observations, separate from baseline estimates. Scene/reset physics is unmeasured. Compute was verified stopped after a complete local backup, within the $15 cap.

Record. Start at `progress/receipts/06-dsrl-results.md`. The decoder and learner audits retain the failed criteria, and `progress/receipts/06-dsrl-completion/` records private revisions, teardown and billing. Further experiments need a new frozen protocol.